

Assignment: Automated Prompt Optimization for Structured Extraction

1. Background

Karpathy's *autoresearch* project (github.com/karpathy/autoresearch) is one example of a general idea: an artifact (code, a configuration, a prompt) is iteratively rewritten by an LLM under a scoring function and a budget. Karpathy's loop happens to be "propose a mutation, accept if score improves, repeat" — but other regimes (populations, beam search, multi-objective, decomposed sub-optimizers, etc.) fit the same idea.

This assignment applies the same idea to **prompt optimization for structured extraction from documents**. The specific optimization algorithm is left to you.

2. Objective

Build a system that automatically improves the prompt(s) used by an LLM on a measurable task, given:

- A dataset of inputs paired with expected outputs.
- A scoring function over `(predicted_output, expected_output)`.
- A seed prompt.
- A budget (iterations, tokens, dollars, wall-clock — your choice of which apply).

Within the budget, the system must produce final prompt(s) that score better than the seed on a held-out test split.

Decisions left to you include — but are not limited to:

- The optimization algorithm (greedy accept/reject, population, beam, Pareto over multiple objectives, multi-stage refinement, decomposition by subtree, ...).
- The structure of the artifact being optimized (a single prompt, prompt + few-shot examples, a chain of prompts, an ensemble, ...).
- The number and type of LLM roles involved (e.g. a single LLM doing everything, separate models for producing outputs vs. proposing prompts, additional roles such as a critic or a judge).
- How evaluation results are summarized and presented to whatever component proposes the next prompt(s).

The specific task used in this assignment is given in Section 3. The system itself must not be specialised to it.

3. Task specification

3.1 Dataset and schema

The dataset is **ExtractBench** (github.com/ContextualAI/extract-bench, MIT). All five PDF → JSON schemas are available for use:

- `academic/research`
- `finance/10kq`
- `finance/credit_agreement`
- `hiring/resume`
- `sport/sport`

For each schema:

- Documents are PDFs, distributed in the ExtractBench repository.
- The JSON Schema and gold annotations are taken **as-published**. They are not modified, extended, or re-annotated.
- You define your own train / validation / test split for each schema. The split policy must be deterministic, documented in the README, and reproducible from a seed.
- You may not augment the dataset with external documents.

3.2 Scoring

Each ExtractBench schema embeds an `evaluation_config` per field (e.g. `string_exact`, `string_semantic`, `integer_exact`, `number_tolerance`, `array_llm`). The scoring function must honour these per-field policies.

Requirements:

- Per-leaf precision, recall, and F1, aggregated across documents.
- An aggregate score per run, plus a per-subtree breakdown.
- A defined alignment policy for repeated arrays. Document the policy.
- Deterministic where the underlying metric is deterministic. Stochastic metrics (`array_llm`, `string_semantic`) must be invoked with fixed parameters and the results cached per (`prediction`, `gold`) pair.

The scoring function must be independent of the optimization loop.

4. System requirements

4.1 Functional

The system must:

1. Accept the dataset, the seed prompt, the scoring function, the LLM(s) it uses, and the budget as configuration inputs.
2. Run an optimization process that, within the configured budget, returns final prompt(s) that outperform the seed on a held-out test split.
3. Terminate on budget exhaustion.
4. Produce a final report containing the seed and final scores on the test split, and a record of the optimization trajectory.

4.2 Non-functional

- All LLM calls and all optimization decisions are persisted to disk.
- A run that is interrupted must be resumable without losing completed work.
- Configuration changes alone must be sufficient to switch the dataset, the scoring function, and the LLM(s) the system uses. The optimization logic must remain untouched.

5. Evaluation criteria

Submissions are graded on the following dimensions.

5.1 Result quality

- Improvement of the score from seed prompt to final prompt on a held-out test split.
- Stability of improvement across the optimization run.
- A finer-grained breakdown of where improvements occurred, at whatever decomposition the task admits.

5.2 Code quality

- Module boundaries, function size, naming, and absence of dead code.
- Handling of expected failure modes across the prediction, scoring, and external-API layers.
- Test coverage of the scoring function in particular.

5.3 Configurability

The repository will be evaluated by reconfiguring it for additional datasets not used during development. Required: no edits outside configuration files. The datasets used for this evaluation are not disclosed in advance.

5.4 Optimization strategy

The design of the optimization process, including the algorithm chosen, the way evaluation signal is converted into the next proposal, the structure of any prompts used to drive LLM components of the optimizer, and the handling of pathological cases (regressions, stalls, repeated proposals, overfitting to the validation split).

5.5 Observability

- Per-iteration logs of prompts, scores, and per-component breakdowns.

- LLM call records (input, output, cost, latency).
- Tooling to inspect the diff between successive accepted prompts and to identify regressions at a finer-grained level than the aggregate score.

6. Deliverables

1. **Source code** in a Git repository, including a **README.md** (setup and run instructions) and a configuration file demonstrating how to retarget the system to a different bundle.
2. **Video** of 2–3 minutes containing:
 - The provided bundle the system was run against.
 - A walkthrough of the code structure, including the optimization process, the scoring function, and the configuration layer.
 - A representative run, with the score curve.
 - One observation from the work.
3. **Report** (**REPORT.md**, 1–2 pages):
 - Seed-prompt and final-prompt test-set scores, including per-subtree breakdown.
 - Seed prompt and final prompt, or a diff summary.
 - Notable mutations accepted during the run.
 - Limitations of the approach as implemented.

The video should be hosted (YouTube unlisted or Drive) and linked from the README.

7. Constraints

- Any LLM provider may be used in any role. At minimum, the role that consumes the document must support image/PDF input.
- Any library may be used. If a third-party library implements a substantial portion of the loop, the report must describe what was used as-is and what was extended.
- Karpathy's autoresearch repository may be referenced but not copied.
- Use of coding assistants is permitted. The author must be able to explain any line of submitted code.

8. Expectations

- scoring function, baseline prompt, end-to-end loop on a small subset of the provided bundle, configuration layer, persistence, observability, mutation-strategy iteration, full-scale run and final submission with code repo link and recorded video.

9. References

- [karpathy/autoresearch](https://github.com/karpathy/autoresearch)
- [Karpathy. autoresearch announcement](#)
- [ExtractBench](#)